1 Introdução à gestão da engenharia de software

Semana 1 · Folha de exercícios

Nome _____ Data _____

Exercícios

Mostre seu raciocínio. Cada exercício traz uma dica no final: tente primeiro sem ela.

1. Encontre a lacuna Aquecimento · Lição 1

Um cliente diz "deixe o relatório mais rápido". Liste duas perguntas que você faria antes de escrever qualquer código.

Dica: Pergunte o que "mais rápido" significa para ele: segundos, minutos ou outro conjunto de dados?

2. Escreva um requisito testável Prática · Lição 1

Reescreva "o login deve ser seguro" como um requisito com uma condição mensurável.

Dica: Nomeie a condição, o ator e o resultado esperado.

3. Escolha um modelo Aquecimento · Lição 2

Um formulário do governo tem regras legais fixas que não vão mudar por dois anos. Qual modelo de processo se encaixa melhor, e por quê?

Dica: Pense em quanta mudança você espera depois que os requisitos forem assinados.

4. Desenhe seu próprio histórico de linhas de base Prática · Lição 2

Uma equipe constrói um app de chat em três iterações: login, mensagens e notificações. Nomeie a linha de base e o incremento após a iteração 2.

Dica: Uma linha de base é uma versão funcional; o incremento é a diferença em relação à anterior.

5. Faça o inventário dos itens de configuração Aquecimento · Lição 3

Liste cinco itens de configuração de um app móvel que não sejam código-fonte.

Dica: Pense em documentos, testes, configurações de build e versões.

6. Rastreie um bug até uma linha de base Desafio · Lição 3

A produção mostra um bug que não estava no build de teste. Descreva as quatro perguntas da GCS que você responderia para encontrar a causa.

Dica: Identificação, controle de versões, controle de mudanças, contabilidade de status.

7. Corrija a mensagem de commit Aquecimento · Lição 4

Reescreva estas mensagens no estilo imperativo: "stuff", "Fixed the bug.", "Added a new function for sums".

Dica: Comece com um verbo, em minúsculas, sem ponto final, com cerca de 50 caracteres.

8. Divida um commit Prática · Lição 4

Um commit mistura uma correção de bug, uma renomeação e um teste novo. Descreva como você o dividiria em três commits lógicos.

Dica: Prepare juntos os arquivos relacionados e faça o commit de cada grupo por vez.

9. Preveja o status Aquecimento · Lição 5

Você edita a.txt, roda git add a.txt e depois edita a.txt de novo. A segunda edição está preparada? O que o git status mostra?

Dica: A preparação captura o arquivo no momento em que você roda git add.

10. Leia o grafo Prática · Lição 5

Rode git log --oneline --graph --all em uma pasta de teste após o Laboratório 2. Explique em uma frase por que main e feature-sum apontam para o mesmo commit.

Dica: O que acontece quando a branch que recebe o merge não tem commits novos?

11. Resolva pela intenção Desafio · Lição 5

Duas branches mudaram a mesma função: uma renomeou um parâmetro e a outra acrescentou uma validação. Descreva a resolução desejada e os comandos.

Dica: Mantenha as duas intenções, depois prepare e faça commit.

12. Ordene o fluxo de trabalho Aquecimento · Lição 6

Coloque em ordem: merge, abrir pull request, enviar a branch, revisar e aprovar, criar a branch.

Dica: Você não pode abrir um pedido antes de a branch existir no remoto.

13. Recupere-se de um push rejeitado Desafio · Lição 6

Um colega enviou mudanças para a main enquanto você trabalhava. Seu git push foi rejeitado. Escreva a sequência segura de comandos.

Dica: Primeiro fetch, depois integre e só então push. Nunca force.

Verificação rápida

Circule uma resposta.

14. Qual item é um item de configuração na gestão de configuração de software?

A) Um documento de requisitos B) A cafeteira do escritório C) Uma agenda pessoal

15. Qual afirmação sobre o Git está correta?

A) O Git é um sistema de controle de versões distribuído; cada clone contém o histórico completo.

B) O Git não consegue registrar um commit sem uma conexão de internet com o GitHub.

C) Git e GitHub são dois nomes para a mesma ferramenta.

16. Em um processo iterativo e incremental, o que representa a diferença entre duas linhas de base sucessivas?

A) Um incremento B) Um registro de riscos C) Apenas a entrega final

17. Um colega diz que a última versão se comporta de forma incorreta. Qual pergunta você deve fazer primeiro?

A) Quais itens de configuração e versões estão nessa linha de base?

B) Quem tem o notebook mais rápido? C) Podemos apagar o repositório e começar de novo?

18. Você quer que um colega revise sua mudança no login antes de chegar à main. O que você faz?

A) Envio uma branch de funcionalidade e abro um pull request para revisão

B) Envio direto para a main e peço feedback depois C) Mando por e-mail um zip do projeto

19. Duas branches mudaram a mesma linha de app.js. O merge para. O que aconteceu?

A) Um conflito de merge; uma pessoa precisa escolher o conteúdo final B) O Git apagou o repositório

C) O remoto está fora do ar

20. O git push é rejeitado porque o remoto tem commits que você não tem localmente. Qual é a correção mais segura?

A) Rodar git fetch, integrar as mudanças remotas (merge ou rebase) e depois enviar de novo

B) Rodar git push --force para sobrescrever o remoto C) Apagar seu repositório local e clonar de novo

21. O git commit falha com "Author identity unknown". O que você deve fazer?

A) Configurar seu nome e e-mail com git config --global user.name e user.email B) Apagar a pasta .git

C) Fazer commit com git add -f

Respostas

1. Exemplo: "Qual relatório, e quão lento ele está hoje em relação à meta?" e "Quem o usa, e com que frequência?" Cada resposta transforma um desejo vago em um requisito que você pode testar.
2. Exemplo: "Após 5 tentativas de login falhas em 10 minutos, a conta é bloqueada por 15 minutos e o usuário recebe um e-mail." Agora um testador consegue verificá-lo.
3. Um processo sequencial (do tipo cascata) pode servir, porque os requisitos são estáveis e erros iniciais seriam caros. Um processo iterativo também funcionaria se a equipe quiser feedback cedo sobre a interface.
4. A linha de base 2 é login mais mensagens, funcionando e documentada. O incremento é a funcionalidade de mensagens em comparação com a linha de base 1.
5. Exemplos: especificação de requisitos, arquivos de design da interface, plano e scripts de teste, arquivos de configuração de build, notas de versão, manual do usuário.
6. Quais versões exatas estão em produção (identificação)? O que mudou entre o build de teste e a produção (controle de versões)? Quais mudanças foram aprovadas e por quem (controle de mudanças)? O build de produção é o registrado na linha de base (contabilidade de status)?
7. fix login redirect loop | fix bug where … (nomeie o bug) | add sum function. Verbos específicos ajudam os revisores a percorrer o histórico rapidamente.
8. Prepare os arquivos da correção e faça commit; prepare os arquivos renomeados e faça commit; prepare o teste e faça commit. Agora cada commit pode ser revisado ou revertido sozinho.
9. Não: a segunda edição não está preparada. O git status lista a.txt tanto em "Changes to be committed" (a primeira versão) quanto em "Changes not staged for commit" (a edição mais nova).
10. A main não tinha commits novos desde que a feature-sum se separou, então o Git moveu o ponteiro para a frente (fast-forward) em vez de criar um commit de merge.
11. Edite a função para que use o novo nome do parâmetro e inclua a validação. Remova todos os marcadores, rode git add no arquivo e depois git commit para concluir o merge.
12. Criar a branch → enviar a branch → abrir o pull request → revisar e aprovar → fazer o merge.
13. git fetch origin, depois git pull origin main (ou git rebase origin/main), resolva os conflitos, rode os testes e então git push origin main.
14. **A** — Qualquer artefato que faça parte do produto ou de sua documentação pode ser um item de configuração. Os requisitos são um exemplo central.
15. **A** — Cada clone tem o repositório e o histórico completos, então você pode fazer commit sem conexão.
16. **A** — Por definição, a diferença entre duas linhas de base é um incremento.
17. **A** — Antes de depurar, identifique exatamente quais versões foram entregues. A GCS torna isso rastreável.
18. **A** — Uma branch de funcionalidade com pull request permite que os revisores vejam o diff e aprovem antes do merge.
19. **A** — O Git não consegue escolher entre duas edições diferentes das mesmas linhas, então marca o conflito para você resolver.
20. **A** — Integre primeiro o trabalho remoto. Um push forçado pode apagar os commits de um colega e deve ser evitado em branches compartilhadas.
21. **A** — O Git guarda o nome e o e-mail do autor em cada commit, então os dois precisam ser configurados primeiro.